Automatisierungs-Möglichkeiten beim Legacy-Reengineering

05.11.2021 BAT Bern

Andres Koch, El. Ing HTL, M. Math (CS)
Object Engineering GmbH
andres.koch@objeng.ch – www.objeng.ch

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Agenda

- Applikations-Modernisierung
  - Redesign (Modularisierung, Architektur)
  - Technologie Transformation
- Was kann mit Automatisierung realistisch gemacht werden?
  - Refactoring
  - Reengineering
- Examples
- Summary

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Fragen zum Beantworten

- In welchen Umständen ist Software-Modernisierung besser als Wegwerfen?
- Wann macht automatisiertes Refactoring oder Reengineering Sinn?
- Kann generierter Code danach manuell gewartet werden?

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Modernisierung von Applikationen

- Verschiedene Strategien
  - Wirf es weg und mach es neu!
  - Architektur-Erneuerung und Überarbeiten des Designs inkl. Modularisierung und Komponenten-Redesign
  - Technologie-Transformation
  - Code-Refaktorierung

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Schritte dorthin

1. Wissen was man hat → Analyse
2. Wissen was man will

   → Wissen was man **nicht** (mehr) **will**
3. Wissen wie man es am Besten löst → Architektur, Design, Refactoring

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Ursprungs- und Ziel-Architektur

Monolithisches Design

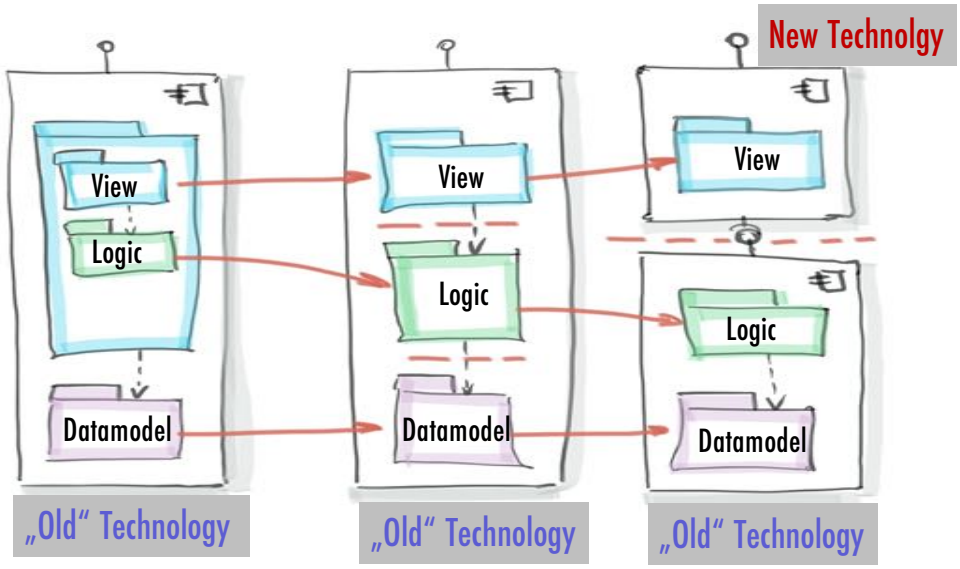
- Rich Clients
- Webserver Container (Tomcat, JEE a.o. with JSP, JSF)
- Application Servers (J EE)

→ Komponenten-basierte Architekturen

- Service based Architecture
- Event driven Architecture
- Microservice Architecture

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Migration auf Umwegen



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Architektur und Redesign

- Komponenten-basierte Architektur ist in der Regel sinnvoll
- Operation → Modul → Komponente (Service) → System
- Service → Funktion + Schnittstelle + Message + Protokoll →ADT/Objekt
- In naher Zukunft werden wir uns mit den Nachwehen des Microservice-Hypes beschäftigen
  → micro-service → nano-service → serverless applications

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Technologie Transformation

- Transformation der Programmiersprachen
  - Programmiersprache X → Y (C++ → Java, C#)
  - Sprachen-Transformation
  - Bibliotheks-Refactoring (Verschiedene Open Source Komponenten mit unterschiedlichen Versionen)
- Datenbank Transformation
  - ORM Transformation
- Transformation der Benutzeroberflächen
  - Rich Client → Web-Client (verschiedene Paradigmas und Frameworks)
  - JSP, JSF, ActiveX → *Single Page Application*

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Wie kann man vorgehen?

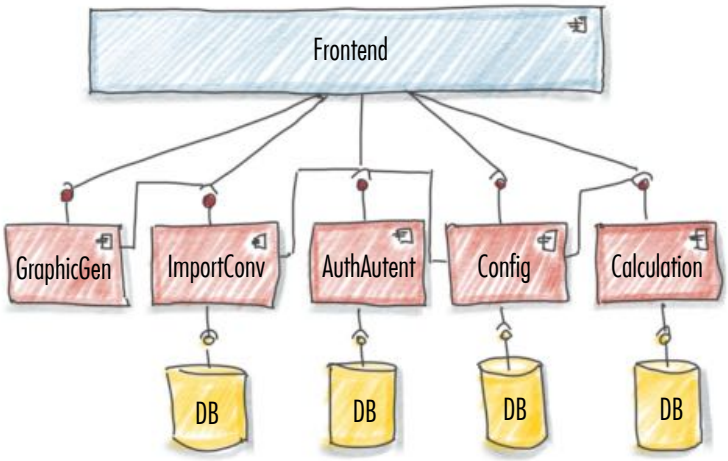
1. Analysieren des bestehenden Systems und Software → übergreifende Abhängigkeiten
   - auf System-Ebene (Komponenten, Konfigurationen, Metadaten)
   - auf Code-Ebene
   - auf Daten-Ebene
2. Zielarchitektur definieren
   - Struktur und Informations-Fluss
   - Technologie definieren (Frameworks, Sprachen u.a.)
3. Reengineering von Code, Komponenten und anderen Artefakten
   - Neuer Code erstellen, Refaktorierung von bestehendem Code oder beides

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Analysieren
manuell oder automatisiert?

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

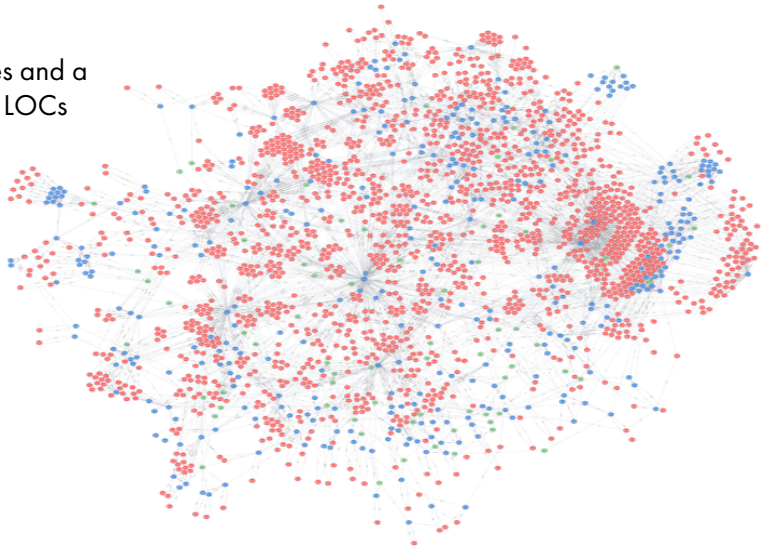
Studentenprojekt



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Kann man die Komponenten sehen?

Project with 5 services and a codebase ~ 23'000 LOCs

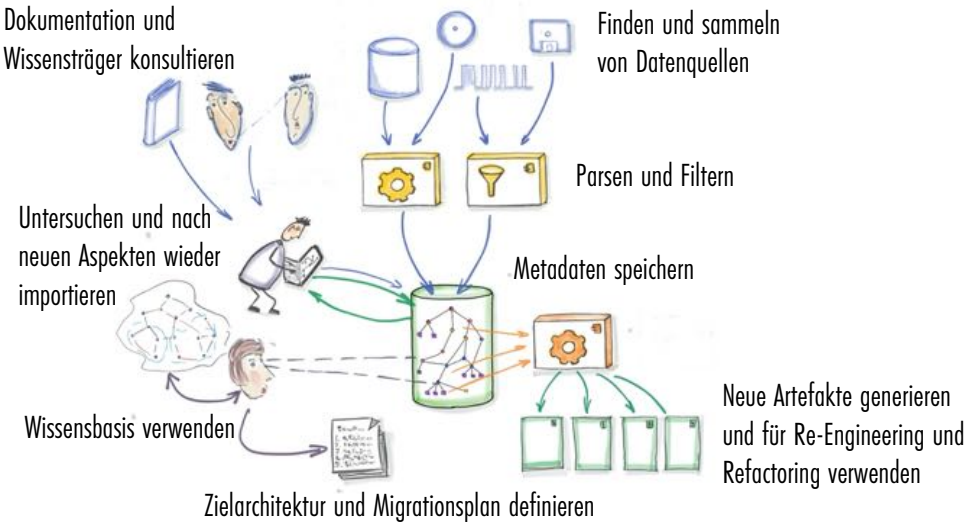


Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Metadaten im Zentrum

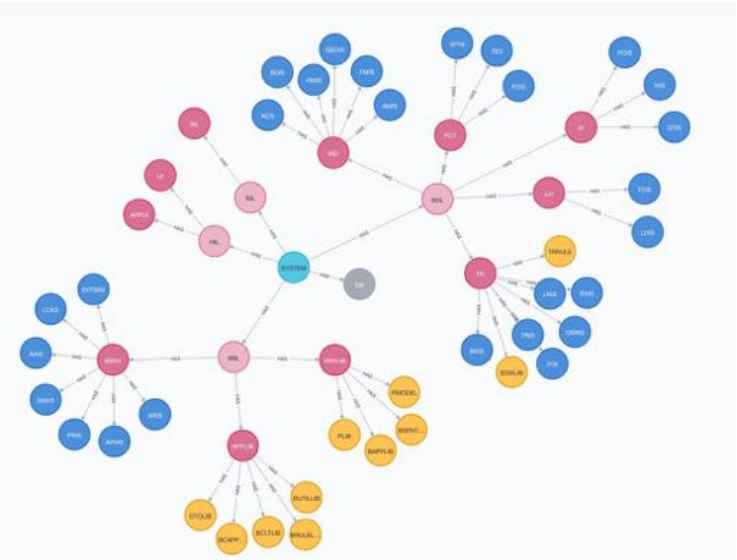
Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Re-Engineering Process



Copyright 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Metadata als Graphen



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Visualisierung

Zweck:

- Visualisierung von komprimierten Aspekten der Wissensbasis ähnlich wie bei Big Data

Methode:

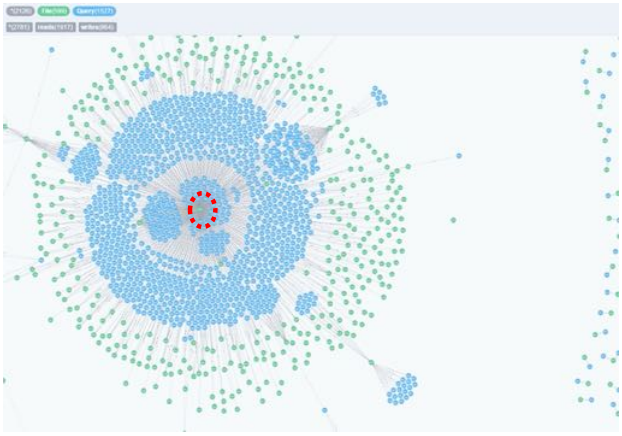
- Artefakte importieren und für spätere Darstellung klassifizieren

Visualisierung

- Selektieren von Bereichen, welche visuell interessant scheinen und von dort runterschrauben zu den Details
- Erkenntnisse für weitere Automatisierung nutzen

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Beispiel: AS400 Batch-Application



- Batch-Applikation für eine missionskritische Marketing- und Verkaufs-Anwendung
- Erwartet, dass 4500 Artefakte dazugehören
- ~100 Tables ~1600 SQL-Scripts
- Externer Lieferant im *phase-out*
- Zweck: Know-How für den weiteren Betrieb und Migration bewahren

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Modularisierung und Kriterien

The effectiveness of a "modularization" is dependent upon the criteria used in dividing the system into modules

David L. Parnas, On The Criteria To Be Used In Decomposing Systems Into Modules, in: Com. of the ACM, December 1972

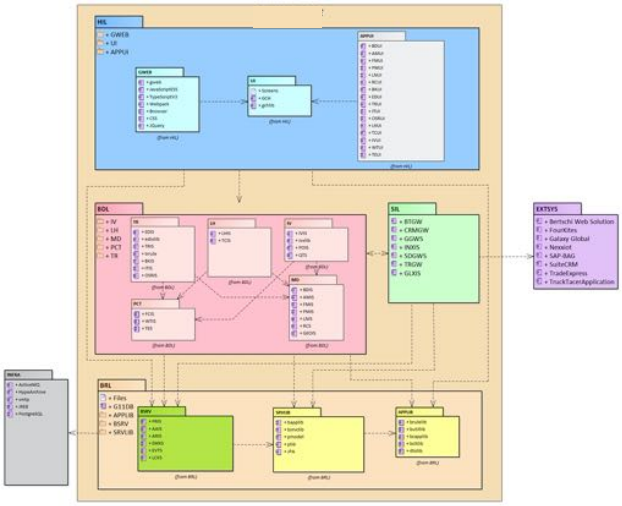
Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

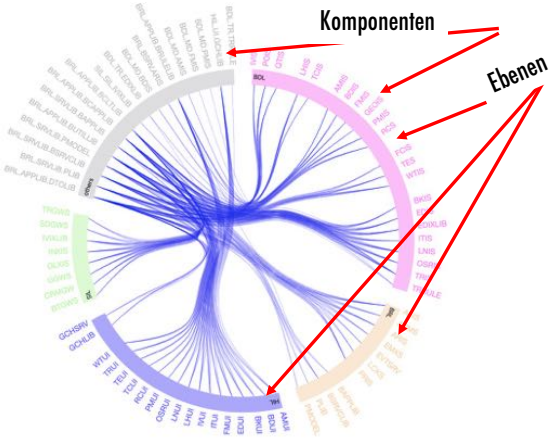
Kriterien-Kandiaten

- File-Benennungen und Namens-Muster
- Prefixes von Modul-Namen in älteren Systemen
- Namensräume z.B. in Import-Statements in mehreren Modulen
- Datenbank-Schemas

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

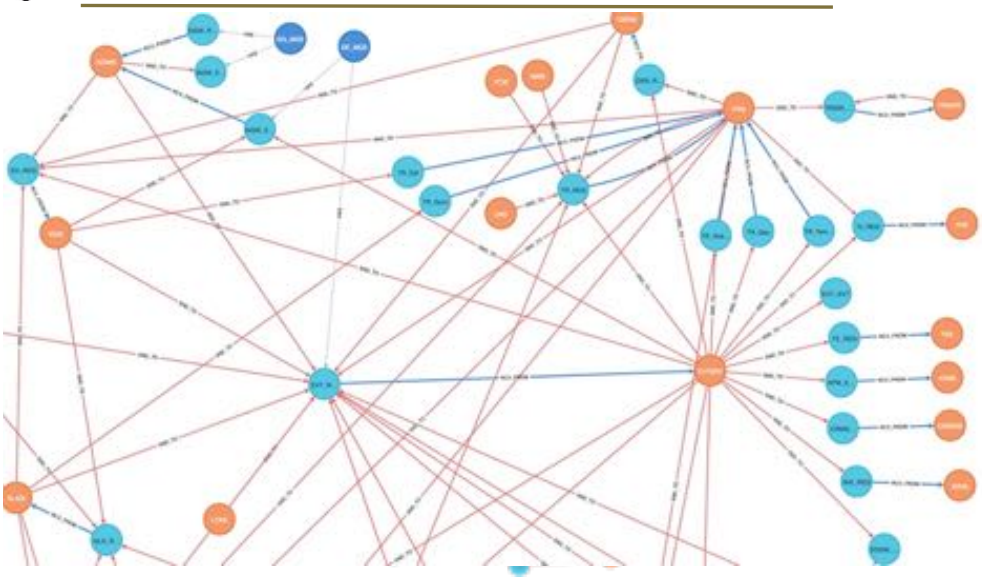
Architektur auf die Wissensbasis abbilden





Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Abhängigkeiten über Interprozess-Kommunikation

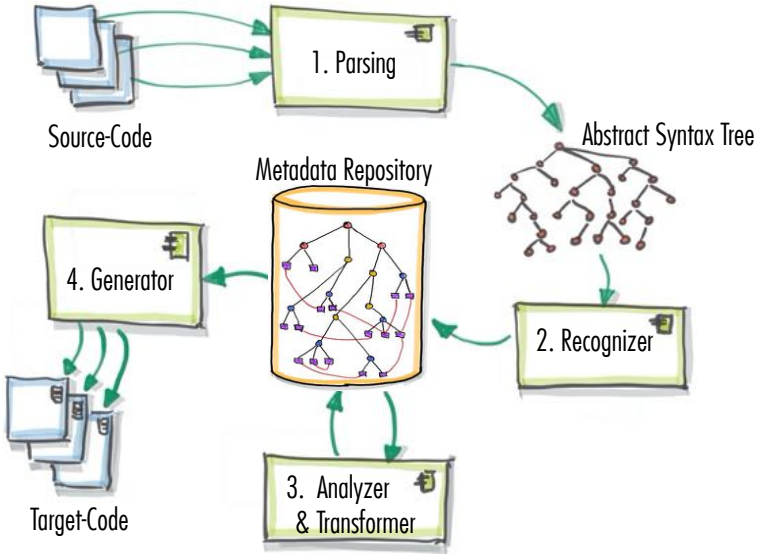


Copyright 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Automatisiertes Re-Engineering and Refactoring

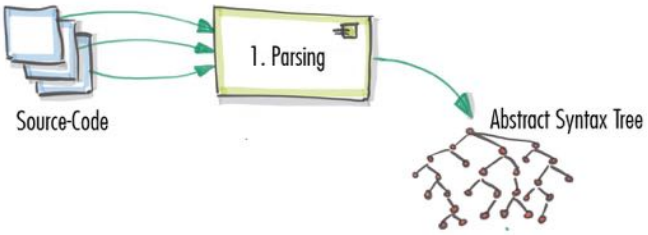
Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Technischer Prozess



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

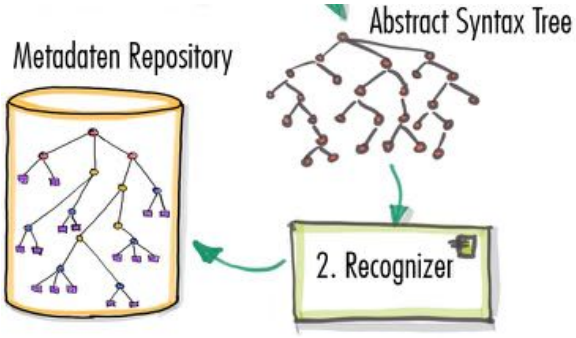
Parsen und importieren von Artefakten



- Parser-Framework (ANTLR) für Code-Artefakte
- Individuelle Parser und Scanner für Konfigurationen und anderer Metadaten

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

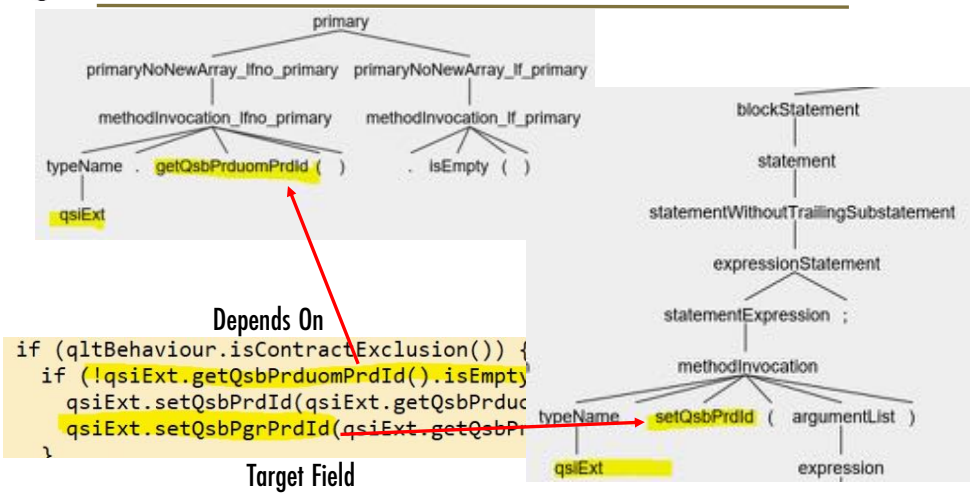
Erkennen und Zuweisen von Kriterien (Tagging)



- Mustererkennung (u.U. pragmatisch und individuell)
- Erkennen von Domänen, Namensräumen u.a.
- Normiertes Metamodel in Wissensbasis aber individuell erweiterbar

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

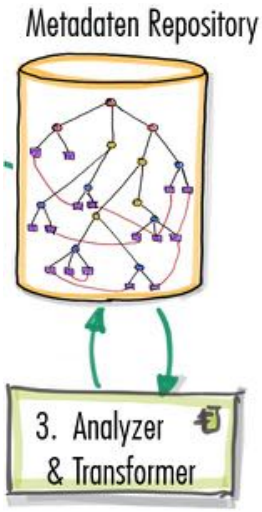
Erkennen von Kriterien im AST



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

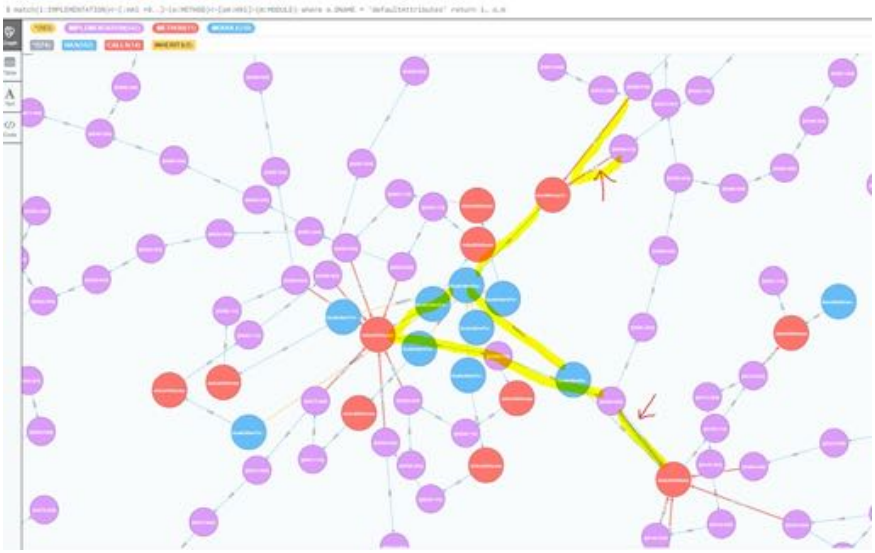
Analyse & Transformation

- Abhängigkeiten einfügen
- Aufrufpfade erkennen
- Gruppen im Graph erkennen
- Erkennen von engen Beziehungen
- Graphen-Algorithmen verwenden für Modularisierung
  - Community Detection Algorithms
  - Closeness Centrality
  - Degree Centrality



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Aufrufpfad-Verfolgung



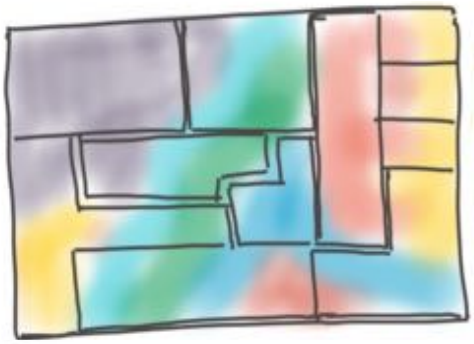
Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

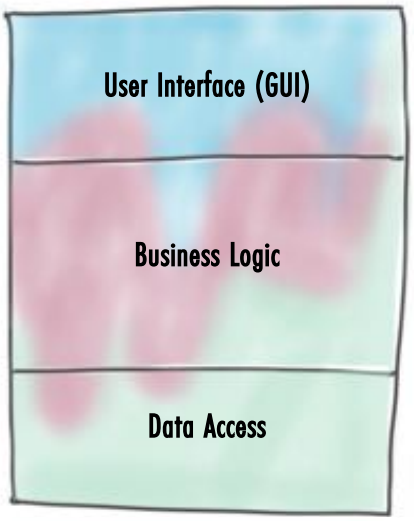
Modulgrenzen finden

- Manuelle Suche nach Schnittstellen in einer grossen Code-Basis ist nicht effizient genug
- Eine voll automatisierte Ermittlung von Schnittstellen bringt kein sinnvolles Resultat
- Die Kombination von Verstand des Menschen und der Geschwindigkeit der Maschine ist erfolgreicher
- Graphen-Algorithmen helfen beim Identifizieren der besten *Soll-Bruchstelle*

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

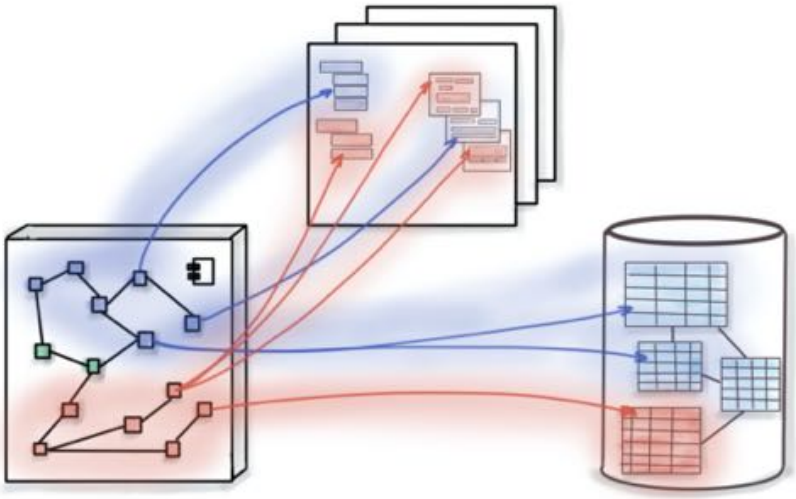
Finden von Modulgrenzen in Monolithen?





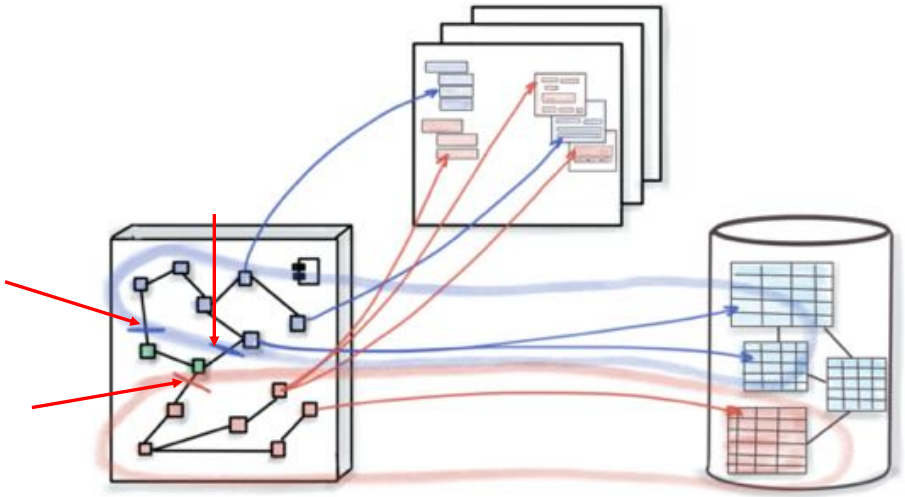
Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Technische Domänen als Kriterium



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

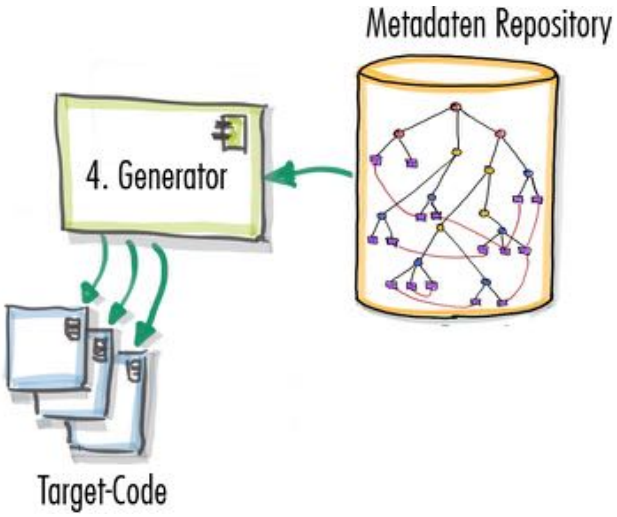
Wo soll der Monolith aufgebrochen werden?



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

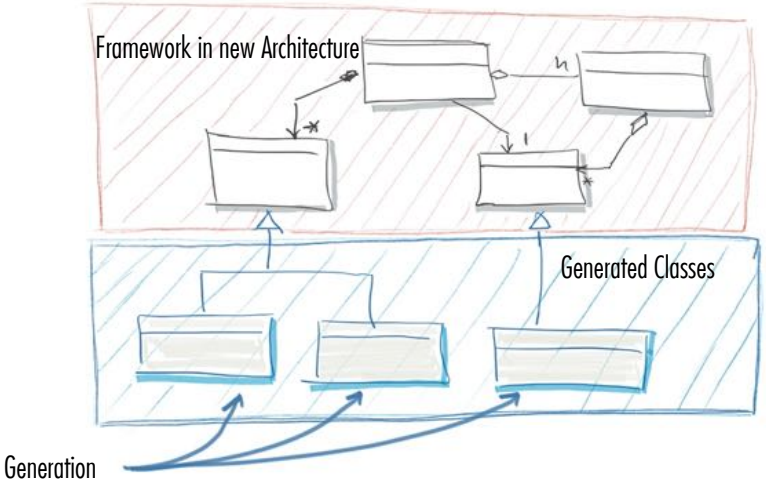
Generierung von neuen Artefacts

- Neue Artefakte generieren
- In vorgegebenes Framework generieren
- Vorgängiges, manuelles Erstellen eines Prototyps bevor man den Generator erstellt



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

In vorgegebenes Framework generieren



Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Beispiel

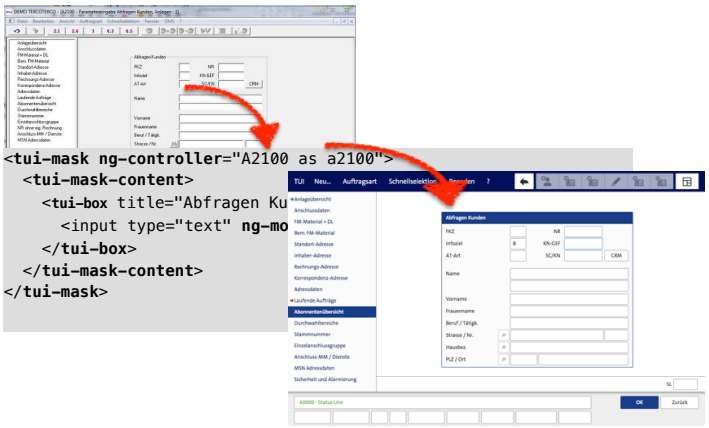
```
class QsbPgrPrdIdDefault
        extends EinkaufItemFieldDefault {

    @Override
    protected Set<QsiFields> createTargetFieldsSet() {
        return EnumSet.of(QSBPGRPRDID);
    }

    @Override
    protected Set<QsiFields> createDependsOnSet() {
        return FieldsMerge.into(EnumSet.of(QSBPRDID, QSBPRDUOMPRDID)).addAll(getGenericDefaulter().getDependsOnFields()).getSet();
    }

    // (01) CH.obj.Application.Its.Quattro.behaviour.purchase.QuattroItemPurchaseContract.defaultAttributes(QuattroItemPurchaseContract.java:3
    // (02) CH.obj.Application.Its.Quattro.behaviour.purchase.QuattroItemPurchaseContract.defaultAttributes(QuattroItemPurchaseContract.java:3
    // (03) CH.obj.Application.Its.Quattro.behaviour.purchase.QuattroItemPurchaseContract.defaultAttributes(QuattroItemPurchaseContract.java:3
    // (04) CH.obj.Application.Its.Quattro.behaviour.purchase.QuattroItemPurchaseContract.defaultAttributes(QuattroItemPurchaseContract.java:4
    // (05) CH.obj.Application.Its.Quattro.behaviour.purchase.QuattroItemPurchaseContract.defaultAttributes(QuattroItemPurchaseContract.java:4
    @Override
    protected void applyInternal(IQsiBuoExt qsiExt, QuattroItemDefaultingContext context, DefaultingTrigger<QsiFields> trigger)
            throws BusinessLogicException {
        getGenericDefaulter().defaultField(qsiExt, context, trigger);

        IQsiBuoExt qsiExt = getExtView(); // (01) [OMANIDff264fc8]
        QLTBEHAVIOUR qltBehaviour = qsiExt.getExtension(QuattroItemExtension.class).getBehaviour(); // (02) [OMANID1b32b780]
        if (qltBehaviour.isContractExclusion()) { // (03) [OMANIDdce14097]
            if (!qsiExt.getQsbPrdUomPrdId().isEmpty()) { // (04) [OMANIDe59a6136]
                qsiExt.setQsbPgrPrdId(qsiExt.getQsbPrdView(IPrdBuoExt.class).getPgrId()); // (05) [OMANID441812a5]
            }
        }
    }
```

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Praxisbeispiele

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Technologie-Migration bei Swisscom AG



- Originally 3270-Applikation from the 80-thies (character based UI)
- Migration to a GUI 1997
  - Visual C++ (MFC)
  - 135 Forms, ~10'000 Fields
  - multilanguage (3)
  - 2'500 Users
- Specialities, application functions
- Various developer teams (generations)
- Patchwork over time

→Migration to a web based single page application

→partially automated

Copyright 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Kosten- und Zeiteinsparung dank Automatisierung

Task	Effort [h]	Effort / Form [h]
Tooling	556	4.2
GUI Framework (Java Script, Angular JS)	770	5.8
Validation of Forms (manual adjustment)	570	4.3
Total	1'896 (74% / 2560)	14.3 (50% Manually)
Quantity Structure		
Forms/Fields	135 Forms / ~10'000 Fields	
Duration	6 Weeks POC + 8 Months Realization	
Cost Savings	50% (Break even 50 = 0%)	
Surplus	Deployment Improvement (~8'000 CHF/ Release) *State of the Art* Technology, Maintenance secured	

Copyright 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Architektur Re-Engineering bei BISON

- Architektur Re-Design mit teilautomatisiertem Prozess: 40-60% Kosteneinsparung
- Code-Fragmente aus 7'000 Klassen extrahiert und 700 neue Klassen generiert
- Generierung von Unit-Test vor der Migration für die nachfolgende Verifikation

Size / Parts	Value
Validators	1'200-1'400
Unit tests	1'200-1'400 gesamt
Reduction of effort validators	min. 1h per validator → 1'200h
Reduction of effort unit tests	min. 2h per unit test → 2'400h
Reduction of project duration	~6 Sprints = ~3 Months
Duration of generator adaption Concept + Realization, Preparation	about 6 month duration Conzept ca. 2 month, Realization 4 month (1 developer OE)

Copyright 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Antworten auf Einstiegsfragen

- In welchen Umständen ist Software-Modernisierung besser als Wegwerfen?
  → grosse eher monolithische Systeme → modularisieren und dann schrittweise modernisieren oder ersetzen
- Wann macht automatisiertes Refactoring oder Reengineering Sinn?
  → Wenn die Zahl von Artefakten sehr gross ist, eine Regelmässigkeit vorhanden ist und der manuelle Prozess zeitintensiv ist
- Kann generierter Code danach manuell gewartet werden?
  → "Wenn ein:e Entwickler:in guten, wartbaren Code schreiben kann, dann kann er/sie auch einen Generator schreiben, der manuell wartbaren Code generiert"

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg

Schlussfolgerung

- Auch wenn gegen eine Modernisierung entschieden wird, sollte das Bestehende als Wissensbasis verwendet werden (Analyse)
- Je grösser die Code-Basis um so grösser der Gewinn aus einer Automatisierung
- Generell: Nicht weniger als 100 K LoCs, erwartete Zeiteinsparung von > 500 h, #Artefake >> 100
- Nicht aufgeben, auch wenn es nicht *perfekt* ist
  → Pragmatische Lösung: Automatisierung kombiniert mit manueller Behandlung (Pareto)
- 1:1 Migration nur für Technologie-Transformation anstreben
- Sich bewusst sein, dass neue Architektur-Muster wieder neuartige Legacy generieren (Bsp. Microservice Architekturen)
- Tip: Wertvolles erhalten und wertloses ersetzen

Copyright © 2021 by Object Engineering GmbH, CH-8142 Uitikon-Waldegg